



Universidade de Coimbra
Faculdade de Ciências e Tecnologia
Departamento de Engenharia Informática

Segurança em Tecnologias da Informação
Project 3 – WSTG and ModSecurity as a WAF

Teachers

António Jorge da Costa Granjal

Team

Carlos João Lopes Mendes, nº2017257272
Filipe Miguel Fonseca dos Santos, nº2017271196

Index

Network Configurations	3
1st Phase	4
1. Tests Performed	4
a. Automated Scan of the Website	4
b. Active Scan (Exploring Effective Policies)	4
c. Add-ons to improve the Test and maximize Threat Identification.....	6
d. Fuzz Attack on the Login Form	6
e. Manual Penetration Test for Logged in Threats.....	7
f. Active Scan (Exploring the Authenticated Area).....	7
g. Additional Tests	8
2. Web Application Penetration Testing - Findings Summary	9
3. Web Application Penetration Testing - Findings Details	10
4.1 Information Gathering	10
4.2 Configuration and Deployment Management Testing	11
4.3 Identity Management Testing	14
4.4 Authentication Testing	15
4.5 Authorization Testing	16
4.6 Session Management Testing	16
4.7 Input Validation Testing	18
4.8 Testing for Error Handling	19
4.9 Testing for Weak Cryptography	20
4.10 Business Logic Testing	20
4.11 Client-side Testing	20
4.12 API Testing	21
2nd Phase.....	22
1. Configurations	22
2. Tests Performed	23
3. Web Application Penetration Testing 2 - Findings Summary.....	24
4. Web Application Penetration Testing 2 - Findings Details [Fixed]	25
5. Web Application Penetration Testing 2 - Findings Details [Not Fixed]	26
6. WAF Performance Evaluation (most important takeaways).....	28
Annexes	29
References	29

Network Configurations

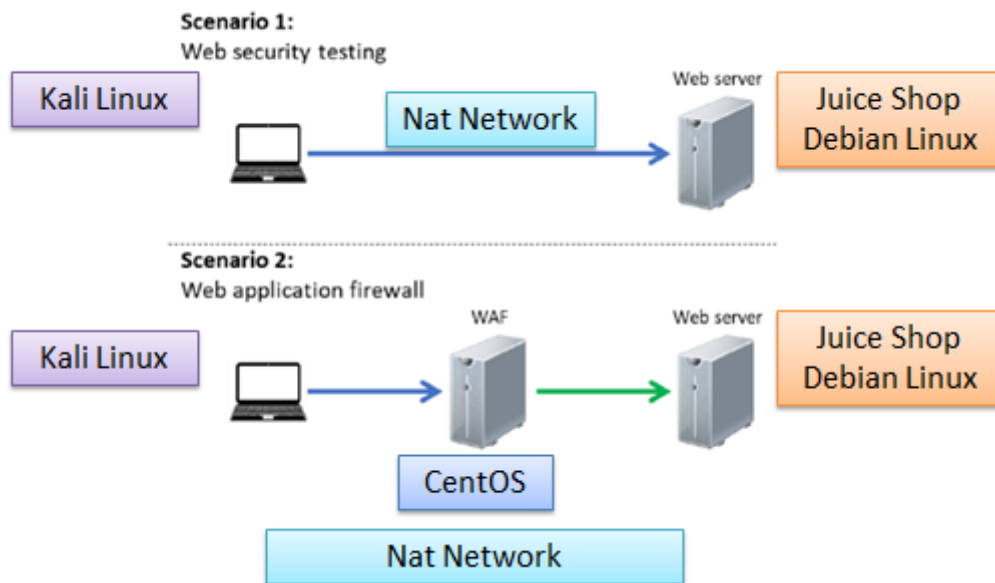


Figure 1: Network Configurations

For both scenarios we use a NAT Network, so that we can connect all machines without having to worry about the IP addresses, since that is not a part of this assignment.

1st Phase

1. Tests Performed

a. Automated Scan of the Website

We started by running an automated scan with the default settings (default policies, no add-ons, no Ajax Spider). However, we quickly realized that, besides the traditional Spider, we should also be using the Ajax Spider, as it can explore links that have been generated [1]. This makes it more powerful, at the cost of being slower.

Using both spiders with the default settings (policies and add-ons) we obtained the following alerts, which we will explain properly in their respective sections, in “2. Web Application Penetration Testing”. This automated scan took roughly 30 minutes.

Name	Risk Level	Number of Instances
Cross-Domain Misconfiguration	Medium	76
Session ID in URL Rewrite	Medium	26
Cookie Without SameSite Attribute	Low	26
Cross-Domain JavaScript Source File Inclusion	Low	6
Private IP Disclosure	Low	2
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	50
X-Content-Type-Options Header Missing	Low	26
Information Disclosure - Suspicious Comments	Informational	7
Timestamp Disclosure - Unix	Informational	59

Figure 2: First Automated Scan (Default Policies and no Add-ons)

b. Active Scan (Exploring Effective Policies)

We started by simply executing an active scan, without changing any policies, so that we could then use it as a baseline. Even with this initial scan, we can already notice alerts that weren't shown before, such as SQL Injection and Cross-Domain Misconfiguration. This active scan took around 30 minutes.

Name	Risk Level	Number of Instances
SQL Injection - SQLite	High	1
Cross-Domain Misconfiguration	Medium	20
Session ID in URL Rewrite	Medium	2
Vulnerable JS Library	Medium	1
Cookie Without SameSite Attribute	Low	2
Cross-Domain JavaScript Source File Inclusion	Low	2
Incomplete or No Cache-control and Pragma HTTP Header Set	Low	23
Private IP Disclosure	Low	1
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	15
X-Content-Type-Options Header Missing	Low	24
Information Disclosure - Suspicious Comments	Informational	5
Timestamp Disclosure - Unix	Informational	382

Figure 3: Active Scan (Default Policies and no Add-ons)

We then searched about the **strength** and **threshold** parameters that appear on the active scan dialogue box. We found out that increasing the strength of a scan increases the number of attacks performed by ZAP, while lowering the threshold causes more things to be considered alerts [2]. It's also important to mention that using the **insane** setting for the strength parameter is usually only considered for very small parts of a system, as it can take a much longer time than the 2nd highest parameter (**high**) [2]. In general, it's recommended to increase this until the point where it starts to have too much of an impact on performance [3]. As for the threshold, lowering it too much can result in receiving too many false positives, while doing the opposite can result in missing important issues [2].

With those ideas in mind, we decided to scan using **strength-high** and **threshold-medium**. Despite taking nearly twice as long to finish than the previous one (1 hour), it didn't manage to find any new alerts, only uncovering more instances of the same ones.

Name	Risk Level	Number of Instances
SQL Injection - SQLite	High	1
Cross-Domain Misconfiguration	Medium	23
Session ID in URL Rewrite	Medium	4
Vulnerable JS Library	Medium	1
Cookie Without SameSite Attribute	Low	4
Cross-Domain JavaScript Source File Inclusion	Low	2
Incomplete or No Cache-control and Pragma HTTP Header Set	Low	23
Private IP Disclosure	Low	1
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	15
X-Content-Type-Options Header Missing	Low	26
Information Disclosure - Suspicious Comments	Informational	5
Timestamp Disclosure - Unix	Informational	381

Figure 4: Active Scan (Strength-High, Threshold-Medium and no Add-ons)

Finally, we tried to change the Input Vectors being used, by selecting all of them. We did this under the assumption that it would increase the number of threats detected. We also made sure to use our standard configuration of strength-high, threshold-medium. However, not only did we not find any new issues, we actually found less than usual. By analysing the sets of attacks performed, we noticed that some of them were skipped. This may have happened due to the program realizing that the attacks would take too many resources, opting not to perform them (which would naturally reduce the number of alerts).

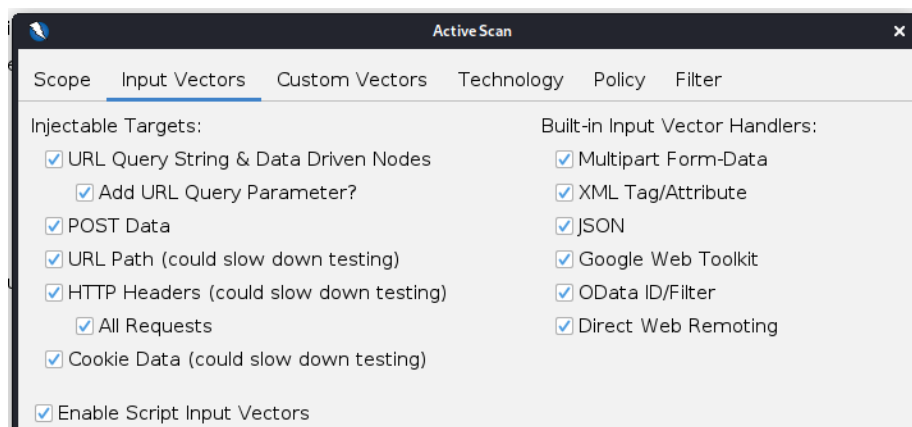


Figure 5: Input Vectors Used in the Active Scan

c. Add-ons to improve the Test and maximize Threat Identification

We used the “Advanced SQLInjection Scanner”. Using this add-on slowed down the speed of the scan tremendously (3 hours). However, we did find a couple of new alerts, as seen below. Most notably, we can see an “Advanced SQL Injection” alert.

Name	Risk Level	Number of Instances
Advanced SQL Injection - AND boolean-based blind - WHERE or HAVING clause	High	1
SQL Injection - SQLite	High	1
Cross-Domain Misconfiguration	Medium	257
CSP: Wildcard Directive	Medium	4
Session ID in URL Rewrite	Medium	28
Vulnerable JS Library	Medium	1
Application Error Disclosure	Low	4
Cookie Without SameSite Attribute	Low	28
Cross-Domain JavaScript Source File Inclusion	Low	336
Incomplete or No Cache-control and Pragma HTTP Header Set	Low	23
Private IP Disclosure	Low	2
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	225
X-Content-Type-Options Header Missing	Low	50
Information Disclosure - Suspicious Comments	Informational	9
Timestamp Disclosure - Unix	Informational	411

Figure 6: Automated Scan (Strength-Medium, Threshold-Medium and the Advanced SQL Injection Add-On)

We also tried using a different version of the “Active Scanner Rules” add-on (alpha version instead of the released version). However, we found no new alerts this way, so it seems to not have much of an impact.

Besides that, we tried the “FuzzDB Files” and “FuzzDB Offensive” add-ons. These ones don’t help with the scan, but rather increase the choices of Fuzzer Files available to use during the next section “d. Fuzz Attack on the Login Form”. These fuzzer files contain multiple attacks, which can be used to test the presence of threats in the web application in multiple scenarios (such as email related attacks).

d. Fuzz Attack on the Login Form

For this part we started by doing an attack based on a string that we already knew would cause problems (‘ **OR true** --). This attack was able to receive an appropriate authentication token and **enter in the application as an admin**, when in reality this shouldn’t be accepted as a valid login, as this is **SQL Injection**.

Afterwards, we started using the default “**File Fuzzers**” instead, which contain multiple attacks.

The first one we tried was “SQL Injection”, leaving every box ticked. Some requests returned a “reflected” status, meaning that **the response contained the given payload**. On top of that, **some responses showed critical fragments of the SQL logic** used in the application (e.g. `SELECT * FROM Users WHERE email = '1 and user_name() = 'dbo' AND password = '0cc175b9c0f1b6a831c399e269772661' AND deletedAt IS NULL`). Other attacks were **not only able to enter in the application, but also showed some variables used in the SQL logic**. (‘ or username is not NULL or username = ’)

Using “**XSS attacks**”, no attacks managed to get an authentication token, but **many got the “reflected” status**, which is bad due to the reasons givens before. Meanwhile, some of

them managed to get internal server error and the **given response contained a somewhat detailed explanation of the error**, including functions names and parameters. This ultimately helps malicious users understand the logic behind the application.

“**Buffer Overflow**” also gives too much information in the response regarding errors.

Using “**XML Injection**”, all the attacks didn’t make it through. However, we did get errors regarding JSON that were too detailed.

As for “**XPath Injection**”, we managed to find scenarios where a 200 OK was issued. On top of that, we also found the same problem regarding errors being too detailed.

We also tried using the “**FuzzDB File Fuzzers**”, to experiment with different attacks. For instance, we used the “**Email**” attacks on the email of the login form. This resulted in too much information related with SQL being shown, due to errors.

e. Manual Penetration Test for Logged in Threats

First of all, someone who manages to enter as an admin can **get access to the email of every account**, through the administration page (196.168.100.6:3000/#/administration).

Messages are not encrypted. If a user gives **sensitive information** (creating an address requires **country, city, zip code, name, mobile number, ...**) and the corresponding message is intercepted, then a malicious user can **receive confidential information** without any protection. This also happens for things like **credit card numbers**, when adding a new one.

By intercepting the PUT request regarding a change in a logged in user, this request can be used to **freely change the wallet** of said user.

Even after logging out, **the authorization token of a session stays valid**. If a malicious user gets a hold of it, then they can **make damaging actions to an account that doesn’t belong to them** (ex: using a put request of changing the wallet, the malicious user can remove money from the target account).

By intercepting a request, a malicious user also has **access to cookies**.

f. Active Scan (Exploring the Authenticated Area)

To understand how these types of scan work, we read the information in [4].

We first entered the website and logged in as a user. From there, we logged out, which made the POST request for the login appear in the “sites” tree. Then, we created a context using that as a basis, so that we could explore threats that happen to logged in users by doing an active scan. For this scan, we decided to use our standard policy configuration (**high strength and medium threshold**).

The only new alert we found this way was “SQL Injection – Authentication Bypass”.

Name	Risk Level	Number of Instances
SQL Injection - Authentication Bypass	High	1
SQL Injection - SQLite	High	1
Cross-Domain Misconfiguration	Medium	26
Session ID in URL Rewrite	Medium	4
Vulnerable JS Library	Medium	1
Cookie Without SameSite Attribute	Low	4
Cross-Domain JavaScript Source File Inclusion	Low	2
Incomplete or No Cache-control and Pragma HTTP Header Set	Low	24
Private IP Disclosure	Low	3
Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)	Low	18
X-Content-Type-Options Header Missing	Low	26

Figure 7: Active Scan (Exploring the Authenticated Area)

g. Additional Tests

Fuzz Attack on "196.168.100.6:3000/" -> trying to find known directories/files - We tried to execute these types of fuzz attacks to see if we could find standard directories or files. However, almost every attack resulted in 200 OK, which in almost every case was a white page. This makes it difficult to truly figure out the directories.

Finding the ftp files page manually - By going to a green link on the "About Us" page of the application, on a new tab, we got to a URL that ended in "/ftp/legal.md". By simply typing "/ftp/" instead, we managed to find a **page containing multiple files that seemed to contain sensitive information, such as receipts, legal documents and general documentation.**

2. Web Application Penetration Testing - Findings Summary

Ref. ID	Title	Risk Level
1	Private IP Disclosure	Low
2	HTTP Headers	Low
3	Specific Files and Folders (/ftp)	High
4	Vulnerable JS Library	Medium
5	Comment Review	Informational
6	Content Security Policy	Medium
7	X-Content-Type-Options	Low
8	Access to Admin Interface	High
9	Cross-Domain Misconfiguration	Medium
10	Cross-Domain JavaScript Source File Inclusion	Low
11	Testing for Default Credentials	Medium
12	Authentication Bypass	High
13	Incomplete or No Cache-control and Pragma HTTP Header Set	Low
14	Bypassing the Authorization Schema (change wallet value)	High
15	Session ID in URL Rewrite	Medium
16	Cookie without SameSite Attribute	Low
17	Reuse of Session Tokens (log out)	High
18	Cross Site Scripting (errors contain too much information)	High
19	SQL Injection	High
20	XML Injection	Medium
21	XPath Injection	High
22	Improper Error Handling	High
23	Sensitive Information Sent via Unencrypted Channels	High
24	Ability to Forge Requests	High

Table 1: Web Application Penetration Testing - Findings Summary

3. Web Application Penetration Testing - Findings Details

For this entire section, we followed the guidelines from [5].

4.1 Information Gathering

4.1.3 Review Webserver Metafiles for Information Leakage

From what we could gather, this exploration of hidden and obfuscated paths is already done automatically by ZAP's spiders. So, simply scanning normally with both spiders enabled can lead to better understanding of the access points and the functionality of the system.

4.1.5 Review Webpage Content for Information Leakage

[Finding 1] Private IP Disclosure - A private IP is shown in some HTTP response bodies. This information can help malicious users to **narrow down the IP range** being used by the system, which can facilitate attacks directed at other internal systems. We consider this a **low** risk threat, since simply knowing this information alone can only help an attacker reduce the time of search, but not much else.

Low (Medium)	Private IP Disclosure
Description	A private IP (such as 10.x.x.x, 172.x.x.x, 192.168.x.x) or an Amazon EC2 private hostname (for example, ip-10-0-56-78) has been found in the HTTP response body. This information might be helpful for further attacks targeting internal systems.
URL	http://192.168.100.6:3000/main-es2015.js
Method	GET
Evidence	192.168.99.100:3000

Figure 8: Private IP Disclosure

4.1.7 Map Execution Paths Through Application

Once again, this is already done automatically by ZAP's spiders, which can discover new URLs. In fact, the AJAX Spider can even find generated URLs [1].

4.1.8 Fingerprint Web Application Framework

[Finding 2] HTTP Headers - Regarding the problem with the "X-Powered-By" HTTP headers, which can give away the framework being used by the application, we found multiple alerts with a "Low" level of risk. All of them were related to the **"Express" framework**. This can be a problem, since by knowing an application uses a certain framework and components, it is possible to easily identify weaknesses in the application that are directly related them. In this case, exploring key vulnerabilities related with "Express" could facilitate attacks on the system. However, knowing this information doesn't help the attacker to bypass the security itself, so we consider this a **low** risk threat.

Low (Medium)	Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s)
Description	The web/application server is leaking information via one or more "X-Powered-By" HTTP response headers. Access to such information may facilitate attackers identifying other frameworks/components your web application is reliant upon and the vulnerabilities such components may be subject to.
URL	http://192.168.100.6:3000/runtime-es2015.js
Method	GET
Evidence	X-Powered-By: Express
URL	http://192.168.100.6:3000/assets/i18n/en.json
Method	GET
Evidence	X-Powered-By: Express

Figure 9: Server Leaks Information via "X-Powered-By" Alert

[Finding 3] Specific Files and Folders (/ftp) – By navigating to the “About Us” page and clicking on a green link, it is possible to reach an “/ftp/legal.md” page. By simply typing “/ftp/” instead, a page showing multiple files with sensitive information is shown instead. In other words, it is possible to get to a specific folder structure that normally shouldn’t be accessible. This can be exploited for attacks. **For instance, we can look at the legal, documentation and receipt files present there and extract confidential information from that**, which is a massive risk for the system. As such, we consider this a **high** risk.

[Finding 4] Vulnerable JS Library – The active scans we performed also found out that the application is using a **vulnerable version of the jquery library (2.2.4)**. This means that the application is susceptible to attacks aiming to exploit the weaknesses of that version. Since the application is using a library with well-known vulnerabilities, we consider this alert **medium**, unlike the one found in [finding 2].

Medium (Medium)	Vulnerable JS Library
Description	The identified library jquery, version 2.2.4 is vulnerable.
URL	https://cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js
Method	GET
Evidence	/2.2.4/jquery.min.js

Figure 10: Vulnerable JS Library

4.2 Configuration and Deployment Management Testing

4.2.2 Test Application Platform Configuration

[Finding 5] Comment Review - Our active scans found informational alerts regarding the use of suspicious comments. The problem with these comments is that they might reveal information regarding the logic used in the application. Attackers can use this type of information to more easily decide which attacks could be harmful. Since these comments seem to only reveal very minor pieces of old code, which is likely not used anymore, we

consider this an **informational** risk. If the comments had more information, then the risk would be considered higher.

Informational (Low)	Information Disclosure - Suspicious Comments
Description	The response appears to contain suspicious comments which may help an attacker. Note: Matches made within script blocks or files are against the entire content not only comments.
URL	https://cdnjs.cloudflare.com/ajax/libs/jquery/2.2.4/jquery.min.js
Method	GET
Evidence	db

Figure 11: Information Disclosure - Suspicious Comments

[Finding 6] Content Security Policy – The automated scan also found issues related with the Content Security Policy. It seems that this policy **isn't configured properly for certain directives (frame-ancestors, form-action)** on the web server, application server, load balancer or in some other component. This means that **resources related with those directives** are being allowed to be **loaded without there being any restrictions [6]**. A malicious user could try to load resources of these types, which is a security concern. However, this isn't quite as threatening as some of the risks that involve confidential information, so we consider it a **medium** risk.

Medium (Medium)	CSP: Wildcard Directive
Description	The following directives either allow wildcard sources (or ancestors), are not defined, or are overly broadly defined: frame-ancestors, form-action The directive(s): frame-ancestors, form-action are among the directives that do not fallback to default-src, missing/excluding them is the same as allowing anything.
URL	http://192.168.100.6:3000/assets/public
Method	GET
Parameter	Content-Security-Policy
Evidence	default-src 'none'

Figure 12: CSP: Wildcard Directive

[Finding 7] X-Content-Type-Options – This header **hasn't been set to "nosniff"**, so it's vulnerable to **MIME-sniffing [7]**. This means that the browser will try to determine the type of content on its own. Because of this, it is possible for **users of old versions of Internet Explorer and Chrome** to force **content to be interpreted and displayed as different type of content than the one declared**. This is relatively minor compared to some other risks, so we only consider it a **low** risk.

Low (Medium)	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
URL	https://content-signature-2.cdn.mozilla.net/chains/pinning-preload.content-signature.mozilla.org-2021-07-01-15-05-55.chain
Method	GET
Parameter	X-Content-Type-Options

Figure 13: X-Content-Type-Options Header Missing

4.2.5 Enumerate Infrastructure and Application Admin Interfaces

[Finding 8] Access to Admin Interface – A user can first access the admin account using **SQL Injection** (this will be further explained in a later section), as well as other methods. From there, it is possible to access the administrator page at “196.168.100.6:3000/#/administration”. In this page, an attacker gets access to sensitive information. **Most notably, he can see the emails of every single account.** This could be used, for instance, to send spam or publicity to all the users of the application.

It’s worth noting that this page can only be accessed by the administrator, giving a 403 error and returning the user to the previous page instead. However, due to the relative ease of accessing the admin account, we consider this a **high** risk.

4.2.8 Test RIA Cross Domain Policy

Cross Domain Policy tries to enforce controlled cross domain access to data and services, using technologies such as Javascript in this case. We found 2 types of alert related with the Cross Domain Policy.

[Finding 9] Cross-Domain Misconfiguration – The problem is that the configuration is setup so that **cross-domain read requests from third-parties domains are allowed.** This only works for the unauthenticated APIs of the system. However, the vulnerabilities of the unauthenticated APIs could be exploited by attackers. For example, it might be possible to access data that uses a different form of protection, besides authentication. However, since it only applies to the unauthenticated part of the application, we consider this a **medium** risk.

Medium (Medium)	Cross-Domain Misconfiguration
Description	Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server
URL	https://cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js
Method	GET
Evidence	Access-Control-Allow-Origin: *

Figure 14: Cross-Domain Misconfigurations

[Finding 10] Cross-Domain JavaScript Source File Inclusion – The issue here is the existence of **pages that include scripts from third-party domains**. Since these scripts come from other domains, the owner of the web application can't fully control who can access them. If the domains supplying the scripts aren't trustworthy they could end up in the hands of attackers, who can then use them to exploit vulnerabilities. Since this type of permission is being given, we assume there is a certain level of trust, so we only consider this a **low** risk.

Low (Medium)	Cross-Domain JavaScript Source File Inclusion
Description	The page includes one or more script files from a third-party domain.
URL	http://192.168.100.6:3000/
Method	GET
Parameter	//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js
Evidence	<script src="//cdnjs.cloudflare.com/ajax/libs/cookieconsent2/3.1.0/cookieconsent.min.js"></script>

Figure 15: Cross-Domain JavaScript Source File Inclusion

4.2.9 Test File Permission

As mentioned in **4.1.8, in the “Specific Files and Folders”, finding 3**, it is possible to access multiple files with sensitive information in the “192.168.100.6:3000/ftp/”, even as an unauthorized user. An attacker can use these files to extract confidential information from the system (which includes legal, documentation, receipts and backup files).

4.3 Identity Management Testing

4.3.1 Test Role Definitions

It's possible to login in the application, so at least 2 roles must exist: **authenticated user and unauthenticated user**. By utilizing SQL Injection (or similar techniques) it is possible to **login as the administrator of the system** (this will be explained in detail in a later section). In terms of permission, only authenticated users can access their profile and do actions related with shopping. Most importantly, only the administrator can access the **administration page** (“192.168.100.6:3000/#/administration”). **On this page, it is possible to view every user's email.**

Notably, **any of the roles can access a page with files that contain sensitive information (legal, documentation, backups and receipts)**, by accessing “192.168.100.6:3000/ftp/”, which is available in a green link on the “About Us” page (this has been explained in a previous section).

Both of these problems have already been addressed by previously mentioned findings.

4.4 Authentication Testing

4.4.2 Testing for Default Credentials

[Finding 11] Testing for Default Credentials - While testing with the usernames present in the administration page, we managed to **enter in the account “demo” using the password “demo”**. Due to its name, we can assume that this account may have been generated to **test the application before its launch**. This is a clear security problem, since not only is there an account that uses **default credentials**, but it also means that **users can have the same username and password**. With this in mind, an attacker that finds a list of users, perhaps through the administration page, can more easily guess the correct passwords. While this can escalate into finding more accounts in the system, besides this default one, we only consider this a **medium** risk, since it doesn't necessarily mean that will be possible.

4.4.4 Testing for Bypassing Authentication Schema

[Finding 12] Authentication Bypass - Using **Fuzz attacks on the login form**, we were able to identify that it is possible to bypass the authentication process, with certain SQL sentences (ex: ' **OR true --**'). In fact, by performing manual tests, we realized that **using such a query as the username, one can enter as administrator of the system**. This can allow an attacker to access the administration page, which allows them to see all usernames and emails. As such, we consider this a **high** risk.

In section “f. Active Scan (Exploring the Authenticated Area)” we found an alert related with authentication bypass, also showing that it is **possible to circumvent the authentication process and access an account**.

High (Medium)	SQL Injection - Authentication Bypass
Description	SQL injection may be possible on a login page, potentially allowing the application's authentication mechanism to be bypassed
URL	http://192.168.100.6:3000/rest/user/login
Method	POST
Parameter	email
Attack	*

Figure 16: SQL Injection - Authentication Bypass

4.4.6 Testing for Browser Cache Weaknesses

[Finding 13] Incomplete or No Cache-control and Pragma HTTP Header Set - Using active scans, we found that the cache-control and pragma HTTP hadn't been correctly set, which allows the browser to cache content. Usually these need to be setup with [no-cache, no-store] and [no-cache] respectively. Otherwise, pages with sensitive information can end up caching confidential data. In this case, an attacker could take advantage of this sensitive information stored on disk, getting access to, for example, a credit card number from a Juice Shop application's user. However, this all depends a lot on the information being cached, which could be completely harmless. So, we only consider this a **low** risk.

Low (Medium)	Incomplete or No Cache-control and Pragma HTTP Header Set
Description	The cache-control and pragma HTTP header have not been set properly or are missing allowing the browser and proxies to cache content.
URL	https://firefox.settings.services.mozilla.com/v1/buckets/main/collections/cfr/changeset?_expected=1620152688322
Method	GET
Parameter	Cache-Control

Figure 17: Incomplete or No Cache-control and Pragma HTTP Header Set

4.5 Authorization Testing

4.5.2 Testing for Bypassing Authorization Schema

[Finding 14] Bypassing the Authorization Schema (change wallet value) - Testing for Horizontal Bypassing Authorization Schema – It is possible to **change the wallet value of another user**, which circumvents the authorization schema. This can be done by intercepting a PUT request related with that user's wallet. From there, it is possible to resend that same request with a different value, changing the amount of money a user has access to. Since authentication tokens stay valid even after logout, this can be done at pretty much any point. This is clearly a high level risk, since it allows the manipulation of other users' money.

4.6 Session Management Testing

4.6.1 Testing for Session Management Schema

[Finding 15] Session ID in URL Rewrite - Session Analysis – Through active scans, we realized that **URL rewrites were being used to keep track of the user session ID**. This session ID is normally stored in a cookie as a precaution, as otherwise it can end up being stored in the browser history or in the server logs. If an attacker gets access to those logs, for instance, they can **obtain the session ID from a certain user**. From there, they can more easily execute attacks that rely on those IDs, such as **impersonation**. However, this relies on the attacker being able to first get access to server logs or browser history, so it is only a **medium** risk in our opinion.

Medium (High)	Session ID in URL Rewrite
Description	URL rewrite is used to track user session ID. The session ID may be disclosed via cross-site referer header. In addition, the session ID might be stored in browser history or server logs.
URL	http://192.168.100.6:3000/socket.io/?EIO=3&transport=websocket&sid=Dyi_leCtJA7aKjUKAAKs
Method	GET
Parameter	sid
Evidence	Dyi_leCtJA7aKjUKAAKs

Figure 18: Session ID in URL Rewrite

4.6.2 Testing for Cookies Attributes

[Finding 16] Cookie without SameSite Attribute – Through the use of active scans we found a problem: there are **cookies without the SameSite Attribute** (typically set as Lax, None or Strict). If this property isn't specified, **the cookie might be sent along cross-site requests**, which can lead to cross-site request forgery, cross-site script inclusion and timing attacks. An attacker can use these types of attacks in order to leak cross-origin information. However, this is the type of threat that by itself isn't a big risk, so we only consider it a **low** one.

Low (Medium)	Cookie Without SameSite Attribute
Description	A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.
URL	http://192.168.100.6:3000/socket.io/?EIO=3&transport=polling&t=Nbsd1z1&sid=jsQmzs_8uBsqaAYAAKr
Method	GET
Parameter	io
Evidence	Set-Cookie: io

Figure 19: Cookie without SameSite Attribute

4.6.2 Testing for Exposed Session Variables

[Finding 17] Reuse of Session Tokens (log out) – **Testing for Encryption & Reuse of Session Tokens Vulnerabilities** – Using manual penetration testing, we found out that it is **possible to reuse session tokens from a different user, even if they have already logged out**. After logging out with a certain user, we used the corresponding session token in order to send a PUT request that **changed that user's wallet**. This type of attack can be used in order to manipulate data from other users, taking advantage of the fact that the session variable is easily accessible and can be **reused freely**. Since this ultimately allows free manipulation of other users' accounts, we consider it a **high** risk.

4.6.6 Testing for Logout Functionality

Once again, as we mentioned on **"4.6.2 Testing for Exposed Session Variables"**, **finding 17**, a session is not terminated properly after a logout. We can verify this, as it is possible to resend requests with the same session id, which can then be exploited with the attacks mentioned before.

4.7 Input Validation Testing

4.7.1 Testing for Reflected Cross Site Scripting

[Finding 18] Cross Site Scripting (errors contain too much information) – Using ZAP’s fuzzing capabilities we were able to execute multiple XSS attacks on the system. None of these managed to get in, but a lot of them got the “reflected” status. That means that the response contains part of the input given. They also showed SQL logic, which shouldn’t be present in these responses.

Also, some of them got internal server errors with detailed explanations (functions and parameters). This can help malicious users to better understand the application.

With all of this information, a malicious user can very easily figure out vulnerabilities in the SQL used in the system, which can heavily aid them in their attacks. As such, we consider this a **high** risk.

4.7.5 Testing for SQL Injection

[Finding 19] SQL Injection – Scans - With some **automatic and active scans**, we managed to find **3 different SQL Injection errors**. One of them has already been explained (“authentication bypass”). Another one is the “Advanced SQL Injection - AND boolean-based blind - WHERE or HAVING clause”, which showed a specific SQL query that could be used **for boolean-based SQL Injection**. Meanwhile, the last one, “SQL Injection - SQLite” demonstrates a possible vulnerability to these attacks, by **abusing generic SQLite queries**.

High (Medium)	Advanced SQL Injection - AND boolean-based blind - WHERE or HAVING clause
Description	A SQL injection may be possible using the attached payload
URL	http://192.168.100.6:3000/rest/products/search?q=%25%27+AND+2374%3D1701+AND+%27%25%27%3D%27
Method	GET
Parameter	q
Attack	%' AND 1124=1124 AND '%='

Figure 20: Advanced SQL Injection - AND boolean-based blind - WHERE or HAVING clause

High (Medium)	SQL Injection - SQLite
Description	SQL injection may be possible.
URL	http://192.168.100.6:3000/rest/products/search?q=%27%28
Method	GET
Parameter	q
Attack	'(
Evidence	SQLITE_ERROR

Figure 21: SQL Injection - SQLite

Fuzzing in the Login Form – By performing Fuzz attacks on the login form, we found that **multiple SQL queries were accepted with a 200 OK response**. In fact, by further testing some of them manually, we realized that it is **possible to enter the admin account**. This can be done by inputting ' **OR true --**. By entering the system this way, anyone can get access to that account, allowing them to execute special functions, such as **viewing the administrator page (which contains all the users and corresponding emails)**. Besides that, there were queries that weren't accepted but returned **very detailed errors**, containing massive amounts of information regarding the **SQL logic of the system**. This can then be exploited by malicious users, who can more clearly understand the vulnerabilities of the SQL used in the system. Due to all of that, we consider this a **high** risk.

4.7.7 Testing for XML Injection

[Finding 20] XML Injection – **Fuzzing in the Login Form** – Using Fuzz attacks on the login form, we came to the conclusion that XML Injection, **while unsuccessful in entering the system, could lead to errors related with JSON being shown in full detail**. An attacker may be able to use this information to their advantage, so we consider this a **medium** risk.

4.7.9 Testing for XPath Injection

[Finding 21] XPath Injection – **Fuzzing in the Login Form** – The use of Fuzz attacks lead to some **requests being accepted with 200 OK**. This can be abused by **malicious users, who can make "valid" requests using this technique**. Besides that, we also found the usual problem of errors giving too much information. Overall, we consider it a **high** risk.

4.8 Testing for Error Handling

4.8.1 Testing for Improper Error Handling

[Finding 22] Improper Error Handling – While experimenting with **Fuzz attacks**, we got **several errors that were far too detailed**. These contained **sensitive information regarding the SQL logic** used by the application, as well as **JSON related information**. This happened for almost every fuzzing attack we tried that generated an error, **including (1) SQL Injection, (2) XSS Injection, (3) XML Injection, (4) XPath Injection, (5) Buffer Overflow, (6) email fuzzing attacks with Fuzz DB**.

This excess of information can be taken advantage of by an attacker, in order to **understand the internal logic of the application better**, which can lead to more powerful attacks. As a result, we consider this a **high** risk.

Also, **ZAP's scans showed that some URLs lead to errors/warnings that contained sensitive information** regarding the way the application behaves. Once again, this means that the errors are too detailed and give attackers too much to work with. This was only considered a low risk by the scans, but due to all the reasons above, we believe that our classification is more fitting.

Low (Medium)	Application Error Disclosure
Description	This page contains an error/warning message that may disclose sensitive information like the location of the file that produced the unhandled exception. This information can be used to launch further attacks against the web application. The alert could be a false positive if the error message is found inside a documentation page.
URL	http://192.168.100.6:3000/rest
Method	GET
Evidence	HTTP/1.1 500 Internal Server Error

Figure 22: Application Error Disclosure

4.9 Testing for Weak Cryptography

4.9.3 Testing for Sensitive Information Sent via Unencrypted Channels

[Finding 23] Sensitive Information Sent via Unencrypted Channels – By exploring **Fuzz attacks**, we realized that **authentication information (credentials) and other kinds of sensitive information (such as the home address) aren't encrypted**. Therefore, if any of such requests are intercepted, a malicious user could have access to sensitive information. This could lead to them accessing other users' accounts, as well as being able to exploit their real life information, making this a **high** risk.

4.10 Business Logic Testing

4.10.2 Test Ability to Forge Requests

[Finding 24] Ability to Forge Requests – While testing **Fuzz attacks**, we discovered that it is possible to **bypass the front end GUI and directly submit our requests through, for example, ZAP**. This allows us to **send data values that are not expected by the application**, which can lead to several problems. For instance, by **intercepting** a request to give a certain rating to Juice Shop, it is possible to **resend the message with a different score**. This includes scores that aren't normally allowed, such as 0. These types of scores are obviously not supported by the business logic of the system, creating potential vulnerabilities to attackers. More importantly, this can also be used to mess with more critical parts of the application, like the users' wallets. Due to this, we consider it a **high** level risk.

4.11 Client-side Testing

4.11.9 Testing for Clickjacking

Using a simple clickjacking method, we found out that the website has built in protection against clickjacking. This doesn't necessarily mean the page is impervious to clickjacking, however, as techniques to circumvent that protection might be possible.

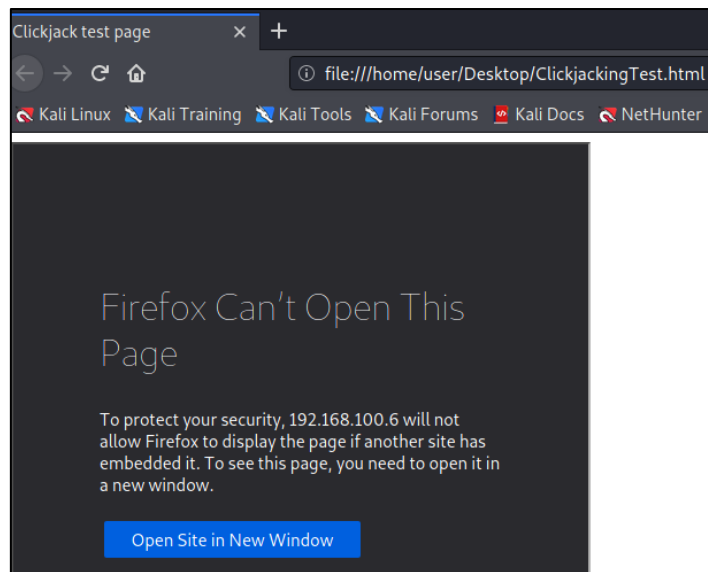


Figure 23: Clickjacking Attempt

```
1 <html>
2   <head>
3     <title>Clickjack test page</title>
4   </head>
5   <body>
6     <iframe src="http://192.168.100.6:3000" width="500" height="500"></iframe>
7   </body>
8 </html>
```

Figure 24: Clickjacking Attempt (corresponding HTML)

4.12 API Testing

4.12.1 Testing GraphQL

As Juice Shop doesn't seem to use GraphQL, we deemed this chapter to be unnecessary for our security report.

2nd Phase

1. Configurations

httpd.conf - In this file we configured the ModSecurity machine as a reverse proxy. To do so, we set Proxy Pass and ProxyPassReverse, so that communications to the reverse proxy are sent to the Juice Shop. We configured it following [8].

```
LoadModule proxy_module modules/mod_proxy.so

LoadModule proxy_http_module modules/mod_proxy_http.so

<VirtualHost *:80>

ProxyPreserveHost On

ProxyPass / http://192.168.100.6:3000/

ProxyPassReverse / http://192.168.100.6:3000/

</VirtualHost>
```

Figure 25: Httpd.conf

mod_security.conf - In order for ModSecurity to accept xml and JSON file, we needed to add the following configuration to this file, as shown in [9].

```
#NEW

SecRule REQUEST_HEADERS:Content-Type "(?:text|application)/xml" \
  "id:'2000004',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=XML"

SecRule REQUEST_HEADERS:Content-Type "application/json" \
  "id:'201425',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"
```

Figure 26: Mod_security.conf, new rules

crs-setup.conf - We also changed the paranoia level to 2, since level 1 alone wasn't covering enough attacks. We chose to not increase it further, due to the possibility of it denying valid requests (false positives) [10].

```
SecAction \  
  "id:900000,\  
  phase:1,\  
  nolog,\  
  pass,\  
  t:none,\  
  setvar:tx.paranoia_level=2"
```

Figure 27: crs-setup.conf

STI_PROJECT_RULES.conf - We created a couple of ModSecurity rules of our own. We deny access to `/ftp`, to prevent unauthorized users from accessing files with sensitive information, such as other users' receipts, backup files and documentation. However, we want users to still be able to read the `legal.md` file, which is supposed to be accessible, so we created a rule for it with an `allow` action. We do the same for `/ftp/order`, so that users can still see their own receipts.

For this part, we followed the official documentation from [11].

```
# Rules list  
  
SecRule REQUEST_URI "/ftp/legal.md" " id:102, allow"  
SecRule REQUEST_URI "/ftp/order" " id:104, allow"  
SecRule REQUEST_URI "/ftp/" " id:101, deny"
```

Figure 28: STI_PROJECT_RULES.conf

2. Tests Performed

We repeated the original tests and verified if the results had changed. The description of these results is in sections 4 and 5.

3. Web Application Penetration Testing 2 - Findings Summary

Ref. ID	Title	Risk Level	Fixed
1	Private IP Disclosure	Low	No
2	HTTP Headers	Low	No
3	Specific Files and Folders (/ftp)	High	Yes
4	Vulnerable JS Library	Medium	No
5	Comment Review	Informational	No
6	Content Security Policy	Medium	No
7	X-Content-Type-Options	Low	No
8	Access to Admin Interface	High	Yes
9	Cross-Domain Misconfiguration	Medium	No
10	Cross-Domain JavaScript Source File Inclusion	Low	No
11	Testing for Default Credentials	Medium	No
12	Authentication Bypass	High	Yes
13	Incomplete or No Cache-control and Pragma HTTP Header Set	Low	No
14	Bypassing the Authorization Schema (change wallet value)	High	Yes
15	Session ID in URL Rewrite	Medium	No
16	Cookie without SameSite Attribute	Low	No
17	Reuse of Session Tokens (log out)	High	No
18	Cross Site Scripting (errors contain too much information)	High	No
19	SQL Injection	High	Yes
20	XML Injection	Medium	Yes
21	XPath Injection	High	Yes
22	Improper Error Handling	High	No
23	Sensitive Information Sent via Unencrypted Channels	High	No
24	Ability to Forge Requests	High	No

Table 2: Web Application Penetration Testing 2 - Findings Summary

4. Web Application Penetration Testing 2 - Findings Details [Fixed]

[Finding 3] Specific Files and Folders (/ftp)

Unauthorized users can no longer access this folder, while still being able to access the legal.md file they are supposed to have access to.

[Finding 8] Access to Admin Interface

Since **SQL Injection, XPath Injection and such all seem to have been fixed**, this is no longer a concern, since in order to enter the administrator page as a malicious user you first need to somehow bypass the authentication (usually done using injection).

[Finding 12] Authentication Bypass

For the reasons explained above, this is **no longer possible**.

[Finding 14] Bypassing the Authorization Schema (change wallet value)

Trying to do this now results in a **“Forbidden” error**. However, it is worth noting that there is a **problem for users trying to change their wallet value in a legitimate way**, which we did not manage to fix.

[Finding 19] SQL Injection

SQL Injection attempts no longer receive the “200 OK” status. This was confirmed by the use of **various Fuzz attacks related with this technique, none of which worked**. The ones that managed to get in before now receive a “Bad Request”. In an automated scan we found 1 instance of SQL injection, but it was related with “/rest/products/search?q=%25”, which only shows information about the products, which is already available in normal circumstances. As such, we did not consider this a problem. Other SQL problems reported in the scans were only related with the issue of showing too much information, which is covered in that specific finding, **[Finding 22] Improper Error Handling**.

[Finding 20] XML Injection

All of the Fuzzing attempts lead to “Bad Request”, which in turn ends up solving the issue of showing too much information in the Internal Errors.

[Finding 21] XPath Injection

The fuzzing attacks all received either “Bad Request” or “Forbidden”.

5. Web Application Penetration Testing 2 - Findings Details [Not Fixed]

[Finding 1] Private IP Disclosure

ModSecurity can't solve this problem, since this is an issue of a **certain response body showing this information**. We could add a specific rule to deny access to the corresponding URL (/main-es2015.js), but without more information about the logic behind making this link available, **we can't know if this link is supposed to be available to normal users**.

[Finding 2] HTTP Headers

ModSecurity can't fix the lack of a proper header setup (cache-control and Pragma HTTP). This is still shown in the scans.

[Finding 4] Vulnerable JS Library

To fix this, **the application itself would need to change the version used for its library**, which is outside the scope of ModSecurity. This is also still shown in the scans.

[Finding 5] Comment Review

This issue is related directly with the code of the application itself, so **it would demand a review of the comments**, with the potentially problematic ones having to be removed manually. Once again, this is shown in the scans.

[Finding 6] Content Security Policy

This risk is actually no longer shown in the scans. However, trying to **manually reproduce the problems** shown in the scans from part 1, **the risk was once again detected**. Since this is a configuration of a policy, it is outside of ModSecurity's scope.

[Finding 7] X-Content-Type-Options

These are options related with the header, so in order to fix this **the configuration of the website itself would need to change**. This risk is still present in the scans.

[Finding 9] Cross-Domain Misconfiguration

This is yet again another configuration, so it's outside of ModSecurity's scope. It still appears in the scans.

[Finding 10] Cross-Domain JavaScript Source File Inclusion

This is a **source file needed by the application**, so it would require the application to be changed. This also still shows up in the scans.

[Finding 11] Testing for Default Credentials

To fix this, **the users with default credentials (ex: “demo”, “demo”) would need to be deleted**, which is something that should be changed with different tools than ModSecurity. If we did this with ModSecurity, the solution would require denying access to that user, which could have unintended consequences.

[Finding 13] Incomplete or No Cache-control and Pragma HTTP Header Set

Since this is related with the headers, it would once again **require a change in the configurations**. This also still shows up in the scans.

[Finding 15] Session ID in URL Rewrite

In order to fix this, a **different method to track the Session ID would need to be used**, such as storing them in cookies. This also still shows up in the scans.

[Finding 16] Cookie without SameSite Attribute

The solution for this **would require properly setting the SameSite Attribute**, which is outside the scope of ModSecurity. We could attempt to block these cookies, but that could affect multiple functionalities of the application. This also still shows up in the scans.

[Finding 17] Reuse of Session Tokens (log out)

This would require **changing the logic that keeps track of the session cookies**, which demands changing the application itself.

[Finding 18] Cross Site Scripting (errors contain too much information)

While **no Cross Site Scripting Fuzzing attempts managed to work** (which was already the case even before the use of ModSecurity), the ones that get **Internal Errors still show too much information**. To fix this, the errors that are shown would need to be altered.

[Finding 22] Improper Error Handling

As mentioned above, **the error messages would need to be changed to contain only generic messages**. This applies to error messages related with every aspect of the tests.

[Finding 23] Sensitive Information Sent via Unencrypted Channels

In order to guarantee that all sensitive information is safe, the data being exchanged would need to be encrypted, which **would require changes in the application logic**. If we use ModSecurity to block every request with sensitive information, then the application **wouldn't work properly anymore**, since those requests are essential to the system.

[Finding 24] Ability to Forge Requests

We **manually tested that it is possible to change another user's username using this technique**. As such, these types of attacks are still possible.

6. WAF Performance Evaluation (most important takeaways)

In general, the WAF seems to deny all the requests that it is supposed to. There are multiple risks that it couldn't fix, but they are all related with configurations or improper error handling (showing too much information in errors) from the application itself, so they are outside the scope of what the ModSecurity WAF is supposed to do.

Annexes

In the folder **Configs** we have all the configuration files mentioned in part 2 of the assignment.

In the folder **ClickjackingAttempt** we have the HTML corresponding with the clickjacking attempt we did.

Finally, the folder **Scans_Reports** contains reports generated from the scans.

References

1. <https://www.zaproxy.org/getting-started/>
2. <https://www.zaproxy.org/docs/desktop/ui/dialogs/scanpolicy/>
3. <https://blog.mozilla.org/security/2013/07/10/how-to-speed-up-owasp-zap-scans/>
4. <https://medium.com/@secureica/authenticated-scan-using-owasp-zap-f0a71d4fe41>
5. https://github.com/OWASP/wstg/tree/master/document/4-Web_Application_Security_Testing
6. https://owasp.org/www-community/controls/Content_Security_Policy
7. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options>
8. <https://linuxtechlab.com/apache-as-reverse-proxy-centos-rhel/>
9. <https://github.com/SpiderLabs/ModSecurity/blob/v3/master/modsecurity.conf-recommended>
10. <https://coreruleset.org/faq/>
11. https://github.com/SpiderLabs/ModSecurity/wiki/Reference-Manual-%28v2.x%29#OWASP_ModSecurity_Core_Rule_Set_CRS_Project